

Research Roadmap: Reproducing & Extending Adam → AMSGrad

Goal: A finished, rigorous, admissions-worthy research artifact — from-scratch implementation of Adam, reconstruction of the AMSGrad non-convergence counterexample, an original extension experiment, and a written report.

How to use this doc: It's organized into 7 phases, each with (a) what to learn, (b) where to learn it for free, (c) what to build, and (d) a clear "done" checkpoint. Phases 0-1 can run in parallel. Don't skip the math phases even if you're tempted to jump to code — the math *is* the differentiator here.

Suggested Pacing (pick the track that matches your week)

Track	Hours/week	Total duration
Light	3-5 hrs	~10-12 weeks
Moderate	8-10 hrs	~6-7 weeks
Intensive	15+ hrs	~4 weeks

Timelines below are given as **effort ranges**, not calendar deadlines — plug in your own pace.

Phase 0 — Math Foundations Audit

(Effort: 1-3 weeks depending on background — you likely know some of this already, so treat it as a checklist, not a course)

You need enough math to **derive**, not just use, these algorithms. Five things to nail down:

Concept	Why you need it	Free resource	Format
Gradients & partial derivatives	Every optimizer update rule is built from these	3Blue1Brown — Essence of Calculus	YouTube series
Linear algebra basics (vectors, norms, matrix ops)	Needed for gradient/moment vector operations	3Blue1Brown — Essence of Linear Algebra ; deeper: MIT OCW 18.06 Linear Algebra (Gilbert Strang)	YouTube + full course
Probability: expectation, variance, concentration inequalities (esp. Hoeffding's)	Needed to understand regret bounds and the theoretical proof structure	MIT OCW 6.041 Probabilistic Systems Analysis ; Khan Academy Statistics & Probability	Full course + interactive
Convexity & optimization theory	Adam/AMSGrad live in online convex optimization framing	Boyd & Vandenberghe — Convex Optimization (free PDF, read Ch. 1–3 and skim Ch. 9)	Free textbook
Online Convex Optimization (the actual framework the papers use)	This is the exact theoretical setting of the AMSGrad proof	Elad Hazan — Introduction to Online Convex Optimization (free, read Ch. 1–3)	Free book (arXiv)

How to read a dense paper without drowning: use the "three-pass method" — first pass (title/abstract/intro/conclusion, 10 min, get the gist), second pass (full read skipping proofs, 30–60 min, understand the setup), third pass (work through the proof with pen and paper, re-deriving each step yourself). Reference: [S. Keshav — "How to Read a Paper"](#) (free PDF).

Done when: you can explain, without notes, what a gradient is, what convexity means, what Hoeffding's inequality says in plain English, and why online convex optimization proofs bound *regret* rather than error.

Phase 1 — Tooling Foundations

(Effort: 3–7 days, run in parallel with Phase 0)

Tool	Why	Free resource
Python fundamentals (if rusty)	Everything is built in Python	Python official tutorial or freeCodeCamp Python course
NumPy	You'll implement everything in raw NumPy, no ML frameworks	NumPy official quickstart + 100 NumPy Exercises (GitHub)
Matplotlib	Convergence plots, loss curves	Matplotlib official tutorials
Jupyter / Google Colab	Free notebook environment, no install, free GPU access if you ever need it	Google Colab (just needs a Google account)
Git & GitHub	Version control + where your portfolio lives	Learn Git Branching (interactive, free) + GitHub Docs
Markdown	For your README	Markdown Guide (10-minute read)
LaTeX (for the final report)	Standard format for research write-ups	Overleaf (free tier, no install) + Learn LaTeX in 30 Minutes

Done when: you can spin up a Colab notebook, write and plot a NumPy function, and push it to a GitHub repo with a README.

Phase 2 — Understand & Implement Adam

(Effort: 1-2 weeks)

Learn:

- Read Kingma & Ba, "[Adam: A Method for Stochastic Optimization](#)" (2014) — free on [arXiv](#) — using the three-pass method from Phase 0.
- Supplementary explainer (optional but helps before the deep read): Andrew Ng's Deep Learning Specialization, "Optimization Algorithms" week — free to **audit** on [Coursera](#) (audit mode = free video access, no certificate).

Build:

1. Vanilla SGD from scratch (NumPy)
2. SGD with Momentum
3. RMSProp
4. Adam (with bias correction — this is the part most tutorials get subtly wrong, so implement it directly from the paper's pseudocode, not from a blog post)

5. Test harness: run all four on 2–3 toy functions (quadratic bowl, Rosenbrock function, one non-convex saddle-point function) and plot convergence

Milestone deliverable: a small optimizer library + a notebook with side-by-side convergence plots + a short written note explaining *why* Adam converges faster than plain SGD on your test functions (in your own words, referencing the moment estimates).

Phase 3 — The Convergence Flaw: Reddi et al. & AMSGrad

(Effort: 1–2 weeks — *this is the mathematical heart of the project*)

Learn:

- Read Reddi, Kale & Kumar, "*On the Convergence of Adam and Beyond*" (2018) — free on [arXiv](#) or [OpenReview \(ICLR 2018\)](#). This is a short, dense paper — expect to need 2–3 passes.
- If the online-learning notation feels unfamiliar, go back to Hazan's free OCO book (Phase 0) Ch. 2 for the regret-bound framework this paper uses.

Build:

1. Reconstruct the paper's synthetic counterexample exactly — it's a simple, explicitly defined sequence of convex functions designed so Adam's effective step size grows in the wrong direction over time.
2. Run vanilla Adam on it and reproduce the qualitative failure (non-convergence / divergence from the optimum) shown in the paper.
3. Implement AMSGrad (the fix: it maintains a running *maximum* of past second-moment estimates instead of an exponential moving average that can shrink inappropriately).
4. Run AMSGrad on the same counterexample and confirm it converges where Adam didn't.

Write (do this alongside the build, not after):

- A derivation walkthrough, in your own words with your own notation, of *why* Adam's second-moment estimate can behave non-monotonically in a way that breaks the convergence guarantee, and *why* AMSGrad's max-based fix resolves it.

Milestone deliverable: a clean, reproducible notebook + plot showing Adam failing and AMSGrad succeeding on the same problem, plus your derivation write-up.

Phase 4 — Original Extension Experiment

(Effort: 1–2 weeks — *this is where your "original contribution" lives*)

This is the single most important phase for making the project more than a replication. Pick **one** concrete hypothesis and test it honestly (a negative result, well-reported, is still a strong result).

Good hypothesis candidates (pick one):

- Does Adam's failure mode from the synthetic example show up — even partially — when training a small real neural network (not just the toy function)?
- Does a newer optimizer variant (e.g., RAdam or AdaBound — both have short, readable papers) actually fix the issue, or does it just hide it in practice?
- How sensitive is the failure to the specific hyperparameters (β_2 in particular) chosen in the counterexample — is it a narrow edge case or does it generalize across a range of settings?

Learn as needed:

- If you want to test on a small real network: keep it in NumPy if possible (fits the "from scratch" narrative), or use minimal PyTorch — [PyTorch's official 60-Minute Blitz tutorial](#) is free and enough to build one small MLP.
- Basic experimental design / statistical reporting: run multiple random seeds, report mean $\pm$ standard deviation or confidence intervals rather than single-run numbers — see [Google's Machine Learning Crash Course, "Generalization" module](#) for a free primer on rigorous evaluation.

Build:

1. Design the controlled experiment (clearly state your hypothesis before running anything)
2. Run it across multiple seeds
3. Report results honestly, including if they don't confirm your hypothesis

Milestone deliverable: a results section with your hypothesis, method, plots, and an honest discussion of what you found (confirmatory or not — both are valid science).

Phase 5 — Research Write-Up

(Effort: 1-2 weeks)

Learn how research papers are structured:

- [Simon Peyton Jones — "How to Write a Great Research Paper"](#) — free talk + slides, one of the best short guides that exists.
- Re-use Keshav's "How to Read a Paper" (Phase 0) — reading it a second time now, from the *writer's* perspective, is genuinely useful.

Structure to follow:

1. Abstract (150–250 words: what you did, what you found)
2. Introduction / Motivation (why Adam's convergence matters, what the flaw is)
3. Background (Adam's math, briefly — assume an informed but non-expert reader)
4. Methodology (your reproduction setup, exactly what you built)
5. Results (reproduction of the counterexample + your extension experiment)
6. Discussion (what it means, limitations, what you'd do with more time)
7. Conclusion
8. References

Tools:

- Write it in Overleaf using a free NeurIPS or ICML-style LaTeX template (searchable inside Overleaf's template gallery) — this alone makes it look like a real paper rather than a homework report.

Milestone deliverable: a 5–10 page PDF report.

Phase 6 — Polish, Repo & Share

(Effort: 3–7 days)

Build:

- Clean GitHub repo: README (what/why/how to run), `requirements.txt`, organized notebooks/scripts folder, your PDF report linked at the top, MIT or similar open license.
- Optional: deploy your convergence plots as an interactive mini-site via free tools like GitHub Pages or Streamlit Community Cloud (both free).

Where this can go beyond your GitHub:

- Personal write-up cross-posted as a blog post (free via GitHub Pages or a free Hashnode/dev.to account) — good for a shareable link in applications.
- Student research journals that accept high-schoolers without institutional affiliation: Journal of Student Research, International Journal of High School Research, Journal of Emerging Investigators — all peer-reviewed, and note that reviewers specifically want to see you go beyond canonical benchmarks (which this project does, via your Phase 4 extension).
- arXiv is technically possible but requires an endorsement from an existing arXiv author in the category — worth asking a teacher or any contact with an academic affiliation if you want this route; not required for the project to be strong.

Milestone deliverable: a public, polished repo + report you'd be comfortable linking in a university application.

Phase 7 (Optional Stretch) — A Second, Smaller Project

If you have time left over, a short second project meaningfully strengthens a "portfolio" narrative (shows range, not just one deep dive). Two good low-compute options from earlier analysis:

Option A — Multi-Armed Bandits (UCB1 & Thompson Sampling)

- Learn: Auer, Cesa-Bianchi & Fischer (2002), free via [this PDF](#) or search "Finite-time Analysis of the Multiarmed Bandit Problem PDF"; Hoeffding's inequality (already covered in Phase 0).
- Build: UCB1 and Thompson Sampling from scratch, verify empirical regret against the theoretical logarithmic bound, apply to a simulated decision problem (e.g., simulated A/B test allocation).
- Effort: ~1 week, pure simulation, no GPU needed.

Option B — Word2Vec for Urdu/Punjabi

- Learn: Mikolov et al. (2013), free on [arXiv](#).
 - Build: skip-gram with negative sampling trained on a public Urdu or Punjabi Wikipedia dump (both freely downloadable from [Wikimedia Dumps](#)), plus your own small hand-built analogy/similarity benchmark since none exists off-the-shelf for these languages.
 - Effort: ~1.5-2 weeks (data cleaning takes real time).
-

Consolidated "Must Learn" Checklist

- ☐ Gradients & partial derivatives
- ☐ Linear algebra (vectors, norms)
- ☐ Expectation, variance, Hoeffding's inequality
- ☐ Convexity basics
- ☐ Online convex optimization framing
- ☐ How to read a dense paper (three-pass method)
- ☐ Python + NumPy fluency
- ☐ Git/GitHub workflow
- ☐ LaTeX basics
- ☐ How to structure a research report

Consolidated "Must Build" Checklist

- ☐ SGD, Momentum, RMSProp, Adam from scratch (NumPy)
 - ☐ Convergence benchmark suite on toy functions
 - ☐ Reconstructed AMSGrad counterexample
 - ☐ AMSGrad implementation + verification
 - ☐ Mathematical derivation write-up (Adam bias correction + AMSGrad fix)
 - ☐ One original extension experiment with a stated hypothesis
 - ☐ Full research report (PDF, 5-10 pages)
 - ☐ Public GitHub repo with README, license, clean structure
 - ☐ (Optional) Second smaller project — bandits or Urdu/Punjabi word2vec
-

A note on using AI tools like Claude while doing this: it's completely fine (and free) to use Claude or similar tools to get concepts explained differently when a resource isn't clicking, or to debug code — just make sure the derivations and analysis in your final write-up are genuinely in your own words and understanding, since that's exactly what makes this project worth more than a tutorial-following exercise.